

Introduction

For this project i wanted to build a working, digital, fun piece that actually uses all of the maths that i was taught in the module and i did not just talk about the maths i wanted to show it. Overall i ended up creating a mini game that i called "Slope Hunter" this is a python styled program which explores gradient descent as a sort of interactive console program with an extremely basic user interface and menu and an experimental "human vs AI" game mode that i had a go at too. I also made this game to explore Jupyter Notebook with smaller experiments and graphs that i could show easily using python code cells and then very easily annotate on using markdown code cells too (with the help of matplotlib and numpy). My overall goal for this project once i figured out what i was set on doing was to use computational thinking (breaking the problem down into many different smaller problems) and also mathematical thinking (functions, derivatives, optimisation trying to implement what i learnt in class) to then build something that shows how gradient descent behaves and also to help me understand this better myself.

Idea Generation

At the start of this project I didn't really know what i wanted to make at all, however i knew i wanted it to be something that would involve real maths and not just some random useless visuals and plotted graphs. I wanted my project to be something that i could actually run and play with and a project that would link into key points like AI and optimisation as these are two things that i was interested in from the beginning of the module that i wanted to tie into this project. When we covered gradient descent in class i struggled to grasp the basics at first but then i realised in my mind it is basically a hill (the curve) that you stand on at a point and you can feel how steep it is (the slope) and you keep on stepping down the hill until you reach the low point. This way of thinking about gradient descent flicked a switch in my mind that made me think that i could turn this into a small fun game instead of a boring formula. My idea then began to develop into level based functions like a simple U shape then expanding into more complicated waves later on, An AI like mode that automatically applies gradient descent and shows the path at the end, A human vs AI like mode where i manually move left or right to try and beat the "AI's" best minimum and an analytics mode in my jupyter notebook where i then compare the different learning rates. This took my ideas from just plotting a graph of a function to a creative applied maths project that i enjoyed creating.

My Code Design and Implementation

When building the code for this project i didnt just throw everything into one long script instead i tried to structure the code in a way that helps me understand it with lots of comments. I struggled a lot with the functions and derivatives for this project so adding comments next to functions with code helped me understand what i was actually coding even though many errors still occurred.

Functions i used for the Maths side

In the code for my project i defined separate functions for each curve and their derivatives for example:

- I used `def f1(x):` and `def f1_prime(x):` for my level 1
- I used `def f2(x):` and `def f2_prime(x):` for my level 2
- Then `def f3(x):` and `def f3_prime(x):` for my level 3

I then stored all of these in a list called `FUNCTIONS = [`

```
FUNCTIONS = [  
    ("Level 1: f(x) = x^2", f1, f1_prime),  
    ("Level 2: f(x) = x^2 - 4x + 5", f2, f2_prime),  
    ("Level 3: f(x) = sin(x) + 0.2x^2", f3, f3_prime),  
]
```

This meant that I could loop over all of them and then let the user choose a level from the basic menu I created instead of just hard coding everything 3 times over.

Gradient Descent Functions

For my gradient descent functions i wrote a single function which was

```
def gradient_descent(f, f_prime, x0, lr=0.1, n_steps=20):
```

 this takes the

- `f` as the function
- `f_prime` as the derivative
- `x0` as the starting x position
- `lr=0.1` as the learning rate
- `n_steps=20` as how many steps to take

Inside this `def gradient_descent` it starts from `x0`, then calculates the gradient using

`f_prime`, then updates the x value using `x = x - lr * g` (g being the gradient)

And finally stores all of the x positions and f(x) values in lists

Menu and Modes creation

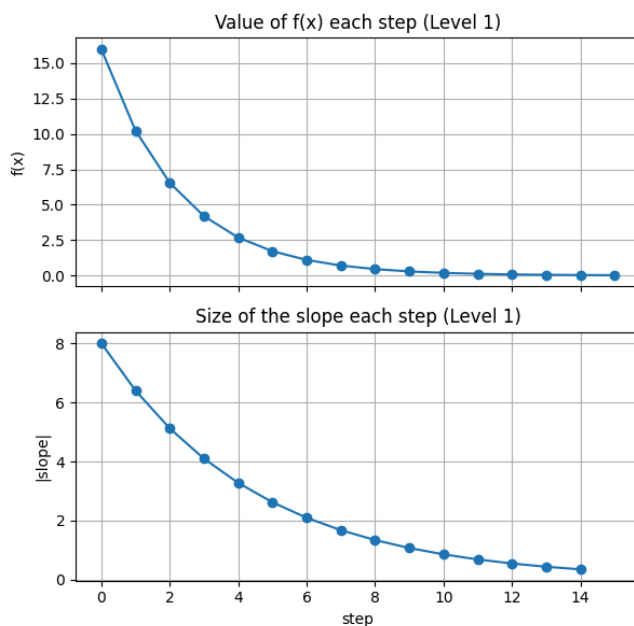
I wrote separate functions for the different modes such as `def ai_demo_mode():` which explains to the user how the game works as it can be very confusing.

```
def ai_demo_mode():
    print("\n=== AI Gradient Descent Demo ===")
    print("Explanation: The computer person is standing on a hill shaped like the f(x) graph in an n shape.")
    print("It can only feel how steep the hill is under its feet (the slope).")
    print("Each step it tries to walk downhill to reach the lower height.\n")
```

Another function i wrote was `def human_vs_ai_mode():` this is the main mode which I experimented with and found to be quite a fun way of learning derivatives. This part lets the try and reach a lower value of $f(x)$ than the "AI/computer" can and in this i explained my way of thinking like the slope is a hill and your trying to get to the bottom using left or right on the x axis.

```
def human_vs_ai_mode():
    print("\n=== Human vs AI: Slope Hunter ===")
    print("your goal is to try and reach a lower value of f(x) than the AI computer.")
    print("you can think of f(x) as the height on a hill and the lower f(x) being closer to the bottom of the hill.")
    print("You need to move left or right along the x axis\n")
```

Then finally I used Jupyter notebook to document how this works visually using the code to display graphs with matplotlib.



Overall this shows the structure of my code and how its not one big pasted chunk. The human vs ai part runs the gradient descent first in the background and then its the users job to find its best $f(x)$. This lets the player type L0.5 or R0.3 And updated the x value based on those moves then keeps track of the users best $f(x)$ so you don't move without knowing .

I explain this in the notebook but this helped me understand the graphs much better with the top plot showing $f(x)$ going down and then flattening out and then bottom plot showing the slope getting smaller and smaller so when the slope is at 0 and the $f(x)$ line is basically flat then thats how you have found its low point.

```
f_vals = np.array(history1["f"])
grad_vals = np.abs(np.array(history1["grad"])) # size of the slope

steps_f = np.arange(len(f_vals))
steps_g = np.arange(len(grad_vals))

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(6, 6))

ax1.plot(steps_f, f_vals, marker="o")
ax1.set_ylabel("f(x)")
ax1.set_title("Value of f(x) each step (Level 1)")
ax1.grid(True)

ax2.plot(steps_g, grad_vals, marker="o")
ax2.set_xlabel("step")
ax2.set_ylabel("|slope|")
ax2.set_title("Size of the slope each step (Level 1)")
ax2.grid(True)

plt.tight_layout()
plt.show()
```

```

PS D:\Maths-Project-Complete> .\SlopeHunter.py
Traceback (most recent call last):
  File "D:\Maths-Project-Complete\SlopeHunter.py", line 22, in <module>
PS D:\Maths-Project-Complete> pip install numpy

```

Here is a screenshot of me running the game, i first had an error saying that numpy wasn't installed when i tried to run it so i quickly installed numpy as the error basically told me to do so.

```

Successfully installed numpy-2.3.5
PS D:\Maths-Project-Complete> .\testenv\Scripts\activate
(testenv) PS D:\Maths-Project-Complete> .\SlopeHunter.py

Slope Hunter
1. AI Gradient Descent Demo
2. Human vs AI Game
3. Analytics: Learning Rate Comparison
4. Quit
Choose an option (1/2/3/4): 

```

After numpy was installed i activated my virtual environment using the code

```

PS D:\Maths-Project-Complete> .\testenv\Scripts\activate

```

in the terminal then i simply ran `.\SlopeHunter.py` to start the game giving me the menu that pops up and asking me to choose an option for the game mode i want to use.

Use of Jupyter Notebook and Virtual Environment

To meet all of the expectations of the module i also thought id show that i understand jupyter notebook, virtual environments and python libraries. I created a Jupyter Note book called

`SlopeHunter_Notebook.ipynb` and then worked inside a virtual environment called

```
> testenv
```

. The addition of Jupyter notebook was a success for this project as it tied in nicely with some explanation and documentation of the game i also learned a lot about the cells with "markdown cells" and how to format headings inside then to make my writing look more presentable and also "python cells" which is where i wrote code to display graphs, show import of `import numpy as np` and `import matplotlib.pyplot as plt`, import my functions from the main game

```
from SlopeHunter import f1, f1_prime, f3, f3_prime, gradient_descent
```

and display gradient descent to the user `history1 = gradient_descent()`.

The reason i created a virtual environment in this project was so it had its own clean setup inside the venv in which i installed numpy, matplotlib and ipykernel. This kept my project separate from the rest of the system . Another things i did was used a small

`requirements.txt` file which lists the main libraries that shows awareness of my project management and also makes it easier for someone else to recreate the environment without me having to tell them what to do.

Problem solving process

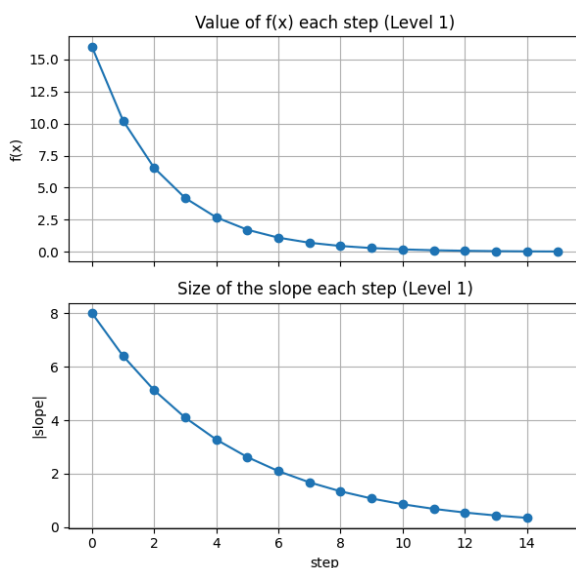
Throughout the process of building this project there were several hiccup points where i had to stop, figure things out and then fix them. Lots of the errors in the project i couldn't figure out myself so had to use different online sources (referenced in the last section of this document) to figure out the right code to implement and make sure it worked.

Environment and Kernel issues

At first activating my virtual environment in the terminal failed because of execution policy settings. This meant i had to learn how to temporarily bypass this to allow activate.ps1 to run. Later on in Jupyter couldn't see my [Slopehunter.py](#) file because the kernel wasnt using the right interpreter. To also fix this i had to select my "testvenv" virtual environment Python to fix the minor error. These problems taught me how python environments work and how they are linked with kernels, this also taught me why Jupyter sometimes cant find my code as i had this error quite a few times throughout my project and finally these errors taught me the importance of picking the right interpreter.

Gradient Descent

When i tested out the gradient descent with different settings i firstly noticed that sometimes it went smoothly down to the minimum of the curve and then other times it jumped around way too much and then other times it barely moved at all. To understand what was happening i used the Jupyter notebook i created to print out all of the x values and $f(x)$ values for each step, plot how $f(x)$ changed over time and plot how the slop got bigger or smaller with each step.



When i looked at both of these graphs it became much easier to understand and see why certain settings worked and others didn't. After this i could clearly spot when the learning rate was too big , too small or just right.

Designing the Human vs AI Mode

When designing the human vs AI mode I also had to think about how to make the mode fair and easy to understand for my user. Some of the decisions I made to tailor this were that the computers run should be hidden but always the same so that the comparison is fair, The human gets the exact same number of moves that the computer gets steps, I also compared the lowest $f(x)$ for each one reached which shows at the end who got closer to the bottom of the hill then finally one of my decisions was to add a small tolerance margin so that if the human gets very close to the AI's best value they're also counted as matching it. This part wasn't just about coding it was also about turning this experiment idea and maths into something that you can actually interact with.

Underlying Mathematical principles

The main ideas I used in this project are quite simple but also pretty effective when they are combined. Firstly I worked with functions which are just the curves on the graph, then I used a simple a simple U shaped curve like $f(x) = x^2$ and then a more interesting wavy curve like $f(x) = \sin(x) + 0.2x^2$ then I used derivatives to find the slope of the curve at a certain point and then the slope tells me which direction that the curve is going in. As an example if we have $f(x) = x^2$ then the derivative is $f'(x) = 2x$ so this means that if x is positive then the curve is going to the right so you move left to go downhill, if the x is negative then the slope is negative too so the curve is going down to the right so you move right and if the x is close to 0 the slope is close to 0 so this area is obviously flat near the minimum. I used all of these ideas directly in my code inside the gradient descent function and then looked at the results using different graphs in Jupyter notebook. For the more complex function $f(x) = \sin(x) + 0.2x^2$ it gets quite a bit harder to then guess what is going to happen just by looking at that equation. Running the gradient descent on this function helped me see how it behaves when the curve has different little small dips and curves which made the maths much clearer.

How derivatives work in my project

In my code for this project I used each function to have a matching derivative function for example $f1_prime(x)$ returns $2*x$ so the derivative is the slope at the x position. Inside my gradient descent function is calculations of $grad = f_prime(x)$ this is the slope at the current x value then I update the position using $x = x - lr * grad$ this means that the AI computer always steps in the opposite direction of the slope going downhill. If the slope is quite big then the step is bigger and if the slope is closer to 0 the step becomes small.

```
for _ in range(n_steps):  
    g = f_prime(x) #  
    grads.append(g)  
    x = x - lr * g #  
    xs.append(x)  
    fs.append(f(x))
```

Here is a screenshot of my gradient Descent code where this happens

External Research

For this project my external research mainly focused on understanding how gradient descent works and how i could use this in python correctly. I used NumPy documentation to understand some of these operations and functions i also used matplotlib documentation to help me plot the visual graphs and then python documentation to understand the virtual environments and how they work overall finally i used some short online explanations of gradient descent to make sure i was understanding this right. Rather than copying someone elses code directly into my project and tweaking it from there i used these sources to understand some of the ideas and then wrote my own code to apply them into this project. Most of my learning came from testing things myself in the jupyter notebook and then seeing how my project worked with different values and graphs.

Throughout this project i also used a mixture of official documentation and online examples to help me complete this and support my understanding of both the maths and python implementation I referred to these:

- Numpy documentation for array operations and maths functions
- Matplotlib documentation for creating and customising different plots
- Python documentation for the virtual environments help
- Online explanations of gradient descent and derivatives to help better my understanding
- Small code examples from online sources such as tutorials and GitHub repositories to understand how to structure gradient descent and derivatives in my code

These examples were used as references to understand logic and structure, rather than copying full code solutions from someone else online. I then wrote and tested my own implementation after tweaking examples from Github and with help from all of the documentation listed above to fit what this project needed.

Understanding of Applied Mathematics

This project helped me understand more how the maths that was very confusing at first in the module can be applied to a real program and be useful. Instead of just working with these random equations that i dont know all that well on paper i used functions and derivatives inside my python code with matplotlib to fully help me understand and implement this. By running lots of gradient descent step by step and visualising the results i could clearly see how the maths worked out rather than thinking about it in my head. As an example changing the learning rate with trial and error showed me how small steps lead to slow but also stable movement while the large steps can overshoot and work unpredictably. Plotting the slope and the value of the function overtime helped me understand what things looked like in front of me instead of just in my head.

Overall i struggled a lot with understanding the maths however after working a lot on this project it made the maths feel much more practical and understandable which helped connect what we did in class in theory to real computer behaviour.

References

- NumPy documentation – <https://numpy.org/doc/>
- Matplotlib documentation – <https://matplotlib.org/>
- Python Software Foundation. *Virtual Environments (venv)* – <https://docs.python.org/3/library/venv.html>
- Python Software Foundation. *The Python Tutorial* – <https://docs.python.org/3/tutorial/>
- General gradient descent explanations and examples from online tutorials and GitHub
- Lecture slides and class notes from the module